

Subject: CS336 Assignment 1, Core Components & LM Pipeline

Date: from 2026/7/22 to 2026/7/23

Contents

Core Components: LM Pipeline

1

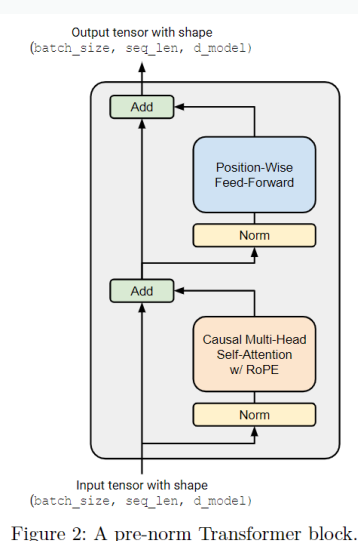
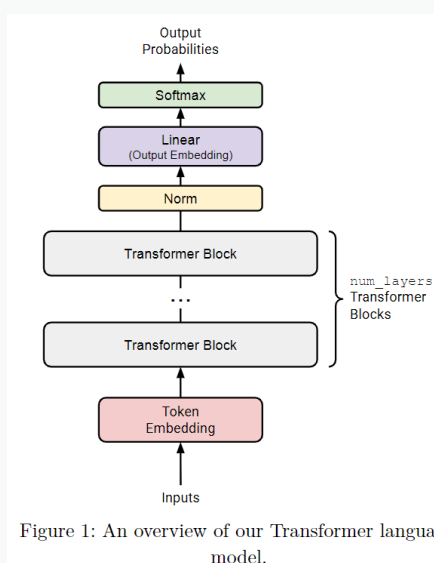
Core Components: LM Pipeline

Overview: 输入与输出

语言模型接收一批整数词元 ID 构成的序列作为输入，形状为 $(batch_size, sequence_length)$ 的 PyTorch 张量；输出是词汇表维度上的概率分布，形状为 $(batch_size, sequence_length, vocab_size)$ ，即每个输入位置预测其后继词元的概率。

训练阶段. 利用后继词元预测结果，计算真实后继词元与模型预测结果之间的交叉熵损失，完成模型优化。

推理阶段. 做文本生成时，只取序列最后一个时间步对应的下一词概率分布，通过贪心取最大概率或概率采样得到新词元；再把它追加到输入序列末尾，循环执行，持续生成后续文本。



输入阶段：从 ID 到张量

1. 原始输入：一批文本经 `Tokenizer` 转换后，变成一个二维整数张量（Tensor）。
 - 形状： $[Batch_Size, Seq_Len]$ ；
 - 例子： $[[332, 545, 612], [866, 959, 1402]]$ （2 条句子，每条 3 个 Token）。
2. 面临的问题：整数 ID 之间没有数学上的相关性（ID 10 和 ID 11 并不代表语义接近），因此需要把它们映射到一个高维的连续空间。

映射阶段：Embedding

- 作用. 将每个整数 ID 转换为一个长度为 d_model 的浮点数向量；
- 数学本质. 一张巨大的查找表，shape 为 $[vocab_size, d_model]$ ；
- 形状变化. $[B, S] \rightarrow [B, S, d_model]$ ；
- 意义. 每个 Token 都有了自己的特征向量；在这个空间里，语义相近的 Token 其向量距离也更近。

核心变换: Transformer Block

数据进入堆叠的 Block, 这是模型进行“思考”和“特征提炼”的核心单元. 一个 Block 包含两个子层: 负责信息交换的注意力层, 以及负责特征加工的前馈网络层.

1. 因果多头自注意力 (Causal Multi-Head Self-Attention)

自注意力的本质是 Q、K、V:

$$\text{SelfAttention}(X) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) V$$

每个 Token 向量通过线性投影生成三个身份:

- **Query (Q)**: 我想找什么? ——当前位置的搜索词;
- **Key (K)**: 我能提供什么? ——所有位置的索引标签;
- **Value (V)**: 我具体的内容是什么? ——所有位置的信息内容.

逻辑: 用 Q 与 K 的点积相似度决定当前 Token 给上下文中其他 Token 分配多少注意力, 最后对 V 加权求和.

2. 多头机制 (Multi-Head)

- 不在 d_{model} 全维度上计算一次注意力, 而是拆成 h 个低维的“头”;
- 允许模型在不同子空间中并行学习, 例如头 1 关注指代关系、头 2 关注时态、头 3 关注句式结构;
- 维度演变: 输入 $[B, S, d_{model}] \rightarrow$ 分别乘 W_Q, W_K, W_V 投影 $\rightarrow$ 拆分为 $[B, S, h, d_{head}]$ 并转置为 $[B, h, S, d_{head}]$ ($d_{head} = d_{model}/h$) $\rightarrow$ 计算注意力后拼接所有头 $\rightarrow$ 经输出线性层 W_O 还原回 $[B, S, d_{model}]$.

3. 因果掩码 (Causal Mask)

- 在形状为 $[S, S]$ 的注意力矩阵中, 把右上角 (未来信息) 填为 $-\infty$;
- 使第 t 个位置做 Softmax 时未来权重降为 0, 严格遵守单向因果律 (只能看过去).

4. 位置编码 (RoPE)

- 在投影得到 Q, K 后, 对其施加复数域或旋转矩阵变换;
- 给 Q, K 注入旋转相位, 使它们的点积结果直接受到“相对距离”的影响.

前馈网络 (FFN / SwiGLU)

注意力层完成空间维度 (Token 与 Token 之间) 的信息交换后, FFN 在通道维度 (Token 自身的特征) 做深度加工.

SwiGLU 结构 (Llama 3 标配). 传统 FFN 是两层线性层中间加一个 ReLU; SwiGLU 引入了门控机制, 计算过程如下:

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}(xW_1) \otimes xW_2)W_3$$

维度操作与升维. 通常把维度从 d_{model} 提升至 d_{ff} , 形状变化为 $[B, S, d_{model}] \rightarrow [B, S, d_{ff}] \rightarrow [B, S, d_{model}]$.

作用. 如果说注意力机制是“把相关的词聚在一起”, 那么 FFN 就是“对聚合后的信息进行逻辑推理和存储”. 大模型中绝大部分的知识和逻辑推理能力都被认为存储在 FFN 的参数中.

输出层：从隐藏特征到词概率

经过 N 层 Block 后，输出张量的形状依然是 $[B, S, d_{model}]$.

1. **最终归一化 (RMSNorm)**: 在输出前做最后一次标准化, 稳定梯度;
2. **LM Head (线性输出层)**: 用一个巨大的 Linear 层把向量维度从 d_{model} 映射回 $vocab_size$, 形状变化为 $[B, S, d_{model}] \rightarrow [B, S, vocab_size]$;
3. **Softmax 概率化**: 在 $vocab_size$ 维度上做 Softmax 得到每个词的概率. 训练时位置 t 的真实标签是 $t + 1$, 模型分配给真实标签的概率越大, 交叉熵损失越小.